Experiment No.: 8

1. Aim: - Write a program to detect Dog image using YOLO Algorithm.

2. Objective: -

- To implement a dog detection system using the YOLO object detection algorithm in Python.
- To identify and locate dogs in images using deep learning techniques.

3. Outcome: -

- Students will be able to apply to YOLO for object detection tasks.
- Students will understand how to detect and visualize objects in images using Python and OpenCV.
- **Software Required:** Python

4. Theory:

The program is to implement a dog detection system using the YOLO (You Only Look Once) object detection algorithm in Python. This system will be capable of identifying and locating dogs within images. It demonstrates the capabilities of deep learning techniques in computer vision applications.

Procedure

Step 1: Install Required Libraries

Make sure you have OpenCV installed. You can install it using pip:

pip install opencv-python numpy

Step 2: Download Required Files

Download the following files and place them in your project directory:

YOLOv3 Weights: yolov3.weights

YOLOv3 Configuration: yolov3.cfg

COCO Names: coco.names

Step 3: Full Code Implementation

Import Libraries

```
import cv2

import numpy as np
```

Load YOLO Files

```
weights_path = "yolov3.weights" (Update the path if necessary)

config_path = "yolov3.cfg" (Update the path if necessary)

names_path = "coco.names" (Update the path if necessary)
```

Step 4: Load the YOLO Network

```
net = cv2.dnn.readNet(weights_path, config_path)
```

Step 5: Load COCO Class Labels

```
with open (names_path, "r") as f:

classes = [line.strip() for line in f.readlines()]
```

Step 6: Load the Image

```
img = cv2.imread("dog.jpg") (Replace with your image path)

height, width, channels = img.shape
```

Step 7: Prepare the Image for YOLO

```
blob = cv2.dnn.blobFromImage(img, 0.00392, (416,416), (0,0,0), True, crop=False)

net.setInput(blob)
```

Step 8: Get Output Layer Names

```
layer_names = net.getLayerNames()

output_layers = [layer_names[i-1] for i in net.getUnconnectedOutLayers()
```

Step 9: Run the Network and Get Detections

```
outs = net.forward(output_layers)
```

Step 10: Initialize Lists for Detection Results

```
class_ids = []
```

```
        confidences = []

        boxes = []
        p1: Install Required Libraries
```

Step 8: Process the Outputs

```
for out in outs:
    for detection in out:
        scores = detection[5:]
        class_id = np.argmax(scores)
        confidence = scores[class_id]

        # Filter for dogs with confidence threshold
        if confidence > 0.5 and classes[class_id] == "dog":
            center_x = int(detection[0] * width)
            center_y = int(detection[1] * height)
            w = int(detection[2] * width)
            h = int(detection[3] * height)

            # Calculate the coordinates of the bounding box
            x = int(center_x - w / 2)
            y = int(center_y - h / 2)

            # Append box details
            boxes.append([x, y, w, h])
            confidences.append(float(confidence))
            class_ids.append(class_id)
```

Step 9: Apply Non-Maximum Suppression

```
indexes = cv2.dnn.NMSBoxes(boxes, confidences, 0.5, 0.4)
```

Step 10: Draw Bounding Boxes on the Image

```
for i in range(len(boxes)):
    if i in indexes:
        x, y, w, h = boxes[i]
        label = str(classes[class_ids[i]])
        confidence = confidences[i]

        color = (0, 255, 0)  # Green color for the bounding box

        cv2.rectangle(img, (x, y), (x + w, y + h), color, 2)
        cv2.putText(img, f"{label} {confidence:.2f}", (x, y - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)
```

Step 11: Display the Output Image

```
cv2.imshow("Detected Dog", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

5. Conclusion: -

In conclusion, this project successfully demonstrates the implementation of a dog detection system using the YOLO (You Only Look Once) algorithm in Python. The system accurately identifies and localizes dogs within images. By leveraging the power of deep learning and computer vision, the project explores image preprocessing, integration of a pre-trained YOLO model, and visualization of detection results.

The results highlight the effectiveness of the YOLO framework in real-time object detection tasks. It also shows potential applications in various domains such as surveillance and wildlife monitoring. This project enhances the understanding of machine learning and object detection techniques. It also provides a foundation for further exploration of advanced object detection and recognition methods.

6. Viva Question

1. What is object detection in computer vision?
2. What is YOLO and how does it work?
3. What does YOLO stand for?
4. What are the advantages of YOLO compared to traditional object detection methods?
5. What is the role of the COCO dataset in YOLO?

7. References

1.J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, You Only Look Once: Unified, Real-Time Object Detection, 2016.
Website: https://pjreddie.com/darknet/yolo/
2.OpenCV Documentation, Object Detection using Deep Learning.
Website: https://docs.opencv.org/
3.J. Redmon and A. Farhadi, YOLOv3: An Incremental Improvement, 2018.
Website: https://arxiv.org/abs/1804.02767
4.TensorFlow / Deep Learning Tutorials, Object Detection Concepts.
Website: https://www.tensorflow.org/tutorials